Laboratory Tasks

Task 1: Array Input, Sum, Average

```c
#include <stdio.h>

int main () {
int n;
printf("enter the number of elements of the array: \n");
scanf("%d", &n);

int a[n];
for (int i = 0; i < n; i++) {
printf("enter an element: ");
scanf("%d", &a[i]);
}
int s = 0;

for (int i = 0; i < n; i++) {
s=s+a[i];
}

float v = (float)s/n;

for (int i = 0; i < n; i++) {
printf("%d ", a[i]);
}

printf("\nsum of all elements: %d\n", s);
printf("avg of all numbers: %f", v);

return 0;
}

```

```
enter the number of elements of the array:
5
enter an element: 1
enter an element: 2
enter an element: 3
enter an element: 4
enter an element: 5
1 2 3 4 5
sum of all elements: 15
avg of all numbers: 3.000000
Process returned 0 (0x0)   execution time : 4.731 s
Press any key to continue.
```

Task 2: Largest, Smallest, and Their Positions

```c
#include <stdio.h>

int main () {
int n;
printf("enter the number of elements of the array: \n");
scanf("%d", &n);

int a[n];
for (int i = 0; i < n; i++) {
printf("enter an element: ");
scanf("%d", &a[i]);
}

int li = 0;
int si = 0;
int l = 0;
for (int i = 0; i < n; i++) {
    if (a[i] > l) {
        l = a[i];
        li = i;
    }
}

int s = l;
for (int i = 0; i < n; i++) {
    if (a[i] < s) {
        s = a[i];
        si = i;
    }
}

printf("largest and smallest elements are %d and %d \n", l, s);
printf("indices of these elements are %d and %d \n", li, si);
return 0;
}
```

```
enter the number of elements of the array:
4
enter an element: 1
enter an element: 2
enter an element: 3
enter an element: 4
largest and smallest elements are 4 and 1
indices of these elements are 3 and 0

Process returned 0 (0x0)   execution time : 4.602 s
Press any key to continue.
```

Task 3: Sort and Search

```c
#include <stdio.h>

int main () {
int n;
printf("enter the number of elements of the array: \n");
scanf("%d", &n);

int a[n];
for (int i = 0; i < n; i++) {
printf("enter an element: ");
scanf("%d", &a[i]);
}

for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j+1]) {
                a[j] = a[j] + a[j+1];
                a[j+1]=a[j] - a[j+1];
                a[j]=a[j] - a[j+1];
            }
        }
}

printf("the elements of the sorted array are: ");
for (int i = 0; i < n; i++) {
    printf("%d ", a[i]);
}

printf("\n");

int t;
printf("enter a target element: \n");
scanf("%d", &t);

int b = 0;
```

```c
int t;
printf("enter a target element: \n");
scanf("%d", &t);

int b = 0;
int bi = 0;

for (int i = 0; i < n; i++) {
    if (a[i] == t) {
        b = 1;
        bi = i;
    }
}

if (b == 1) {
    printf("element %d found at %d", t, bi);
}

else printf("element not found");

return 0;
}
```

```
enter the number of elements of the array:
5
enter an element: 2
enter an element: 7
enter an element: 4
enter an element: 5
enter an element: 1
the elements of the sorted array are: 1 2 4 5 7
enter a target element:
5
element 5 found at 3
Process returned 0 (0x0)   execution time : 7.897 s
Press any key to continue.
```

Additional Practice Tasks

Task 1: Even or Odd count in Array

```c
#include <stdio.h>

int main () {
int n;
printf("enter the number of elements of the array: \n");
scanf("%d", &n);

int a[n];
for (int i = 0; i < n; i++) {
printf("enter an element: ");
scanf("%d", &a[i]);
}

int e = 0;
int o = 0;

for (int i = 0; i < n; i++) {
    if (a[i] % 2 == 0) {
        e++;
    }

    else o++;
}

printf("the number of even elements are %d and the number of odd elements are %d", e, o);
return 0;
}
```

```
enter the number of elements of the array:
5
enter an element: 2
enter an element: 3
enter an element: 4
enter an element: 5
enter an element: 6
the number of even elements are 3 and the number of odd elements are 2
Process returned 0 (0x0)   execution time : 4.475 s
Press any key to continue.
```

Task 2: Reverse an Array

```c
#include <stdio.h>

int main () {
int n;
printf("enter the number of elements of the array: \n");
scanf("%d", &n);

int a[n];
for (int i = 0; i < n; i++) {
printf("enter an element: ");
scanf("%d", &a[i]);
}

int l = 0;
int u = n - 1;

while (l < u) {
a[l] = a[l] + a[u];
a[u] = a[l] - a[u];
a[l] = a[l] - a[u];
l++;
u--;
}

printf("the reversed array is: ");
for (int i = 0; i < n; i++) {
printf("%d ", a[i]);
}

return 0;
}
```

```
enter the number of elements of the array:
5
enter an element: 2
enter an element: 4
enter an element: 2
enter an element: 7
enter an element: 1
the reversed array is: 1 7 2 4 2
Process returned 0 (0x0)   execution time : 7.043 s
Press any key to continue.
```

Task 3: Duplicate Numbers in an Array

```c
#include <stdio.h>

int main () {
int n;
printf("enter the number of elements of the array: \n");
scanf("%d", &n);

int a[n];
for (int i = 0; i < n; i++) {
printf("enter an element: ");
scanf("%d", &a[i]);
}

for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j+1]) {
                a[j] = a[j] + a[j+1];
                a[j+1]=a[j] - a[j+1];
                a[j]=a[j] - a[j+1];
            }
        }
}
int d = 0;

for (int i = 0; i < n - 1; i++) {
        if (a[i] == a[i+1]) {
            d++;
        }
}

printf("the number of duplicates in an array are: %d", d+1);
return 0;
}
```

```
enter the number of elements of the array:
5
enter an element: 2
enter an element: 2
enter an element: 1
enter an element: 4
enter an element: 2
the number of duplicates in an array are: 3
Process returned 0 (0x0)   execution time : 3.859 s
Press any key to continue.
```

Task 4: Merge Two Arrays

```c
#include <stdio.h>

int main() {
int n;
printf("enter the number of elements of first array: \n");
scanf("%d", &n);

int a[n];
for (int i = 0; i < n; i++) {
printf("enter an element: ");
scanf("%d", &a[i]);
}

int m;
printf("enter the number of elements of second array: \n");
scanf("%d", &m);

int b[m];
for (int i = 0; i < m; i++) {
printf("enter an element: ");
scanf("%d", &b[i]);
    }

int l = n+m;
int r[l];

for (int i = 0; i < n; i++) {
r[i] = a[i];
}

for (int i = 0; i < m; i++) {
r[n+i] = b[i];
}

printf("the merged array is: ");
```

```c

printf("the merged array is: ");
for (int i = 0; i < l; i++) {
printf("%d ", r[i]);
}
printf("\n");

for (int i = 0; i < l - 1; i++) {
        for (int j = 0; j < l - i - 1; j++) {
            if (r[j] > r[j+1]) {
                r[j] = r[j] + r[j+1];
                r[j+1]=r[j] - r[j+1];
                r[j]=r[j] - r[j+1];
            }
        }
}

printf("the elements of the sorted array are: ");
for (int i = 0; i < l; i++) {
    printf("%d ", r[i]);
}
return 0;
}
```

```
enter the number of elements of first array:
3
enter an element: 1
enter an element: 5
enter an element: 3
enter the number of elements of second array:
4
enter an element: 1
enter an element: 6
enter an element: 3
enter an element: 5
the merged array is: 1 5 3 1 6 3 5
the elements of the sorted array are: 1 1 3 3 5 5 6
Process returned 0 (0x0)   execution time : 7.891 s
Press any key to continue.
```

Task 5: Transpose of Two Arrays

```c
#include <stdio.h>

int main () {
int n, m;
printf("enter the dimensions of the matrix: ");
scanf("%d%d", &n, &m);

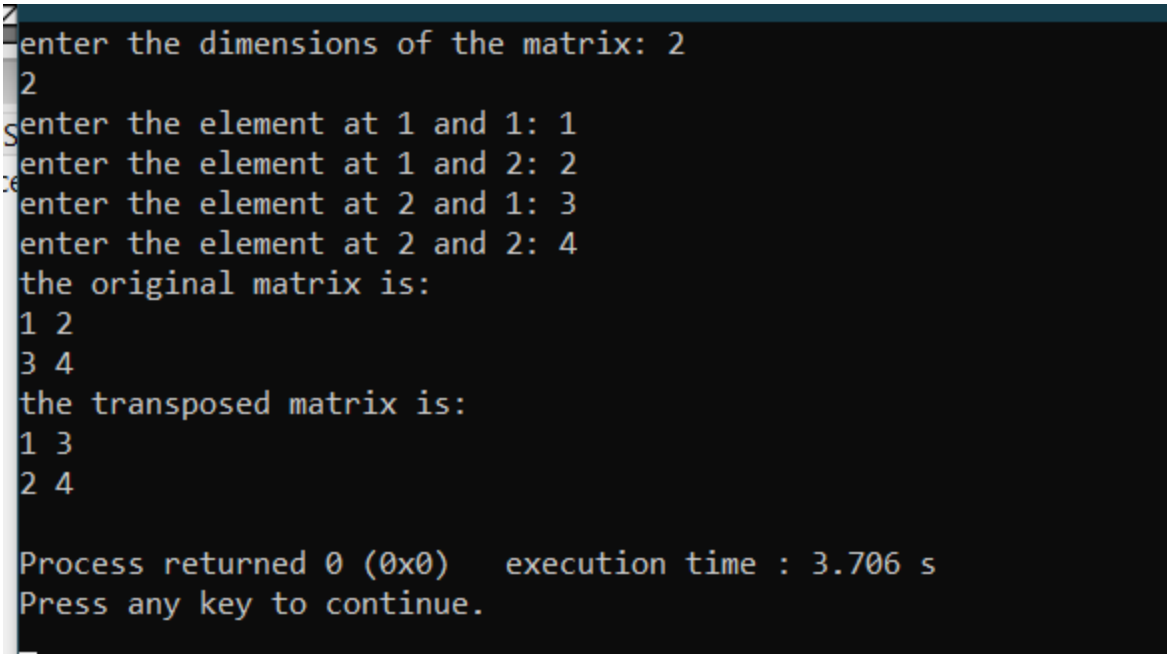
int a[n][m];
int b[m][n];

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        printf("enter the element at %d and %d: ", i+1, j+1);
        scanf("%d", &a[i][j]);
    }
}

printf("the original matrix is: \n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        printf("%d ", a[i][j]);
    }

    printf("\n");
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        b[j][i] = a[i][j];
    }
}
printf("the transposed matrix is: \n");
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", b[i][j]);
    }
    printf("\n");
}
return 0;
}
```


enter the dimensions of the matrix: 2
2
enter the element at 1 and 1: 1
enter the element at 1 and 2: 2
enter the element at 2 and 1: 3
enter the element at 2 and 2: 4
the original matrix is:
1 2
3 4
the transposed matrix is:
1 3
2 4
Process returned 0 (0x0) execution time : 3.706 s
Press any key to continue.

Ex 4

Q.1) An array is a collection of elements of the same data type stored under one name. Unlike individual variables, arrays let us access multiple values using indices.

Q.2) The index represents a memory offset rather than a count.

Q.3) Out-of-bounds causes undefined behaviour, potentially producing garbage values or crashing the program.

Q.4) Because it searches for an element sequentially (one by one linearly).

Q.5) (0 <= i && i<n)

Q.6) A 2D array is stored row by row in a contiguous memory. Iterating through columns inside the outer row loop improves memory access efficiency.

Q.7) Basic C arrays generally require a size known at declaration or a valid runtime size for a variable-length array. Their size determines how much memory is allocated

Q.8) {5, 2, 8, 1}
→ {2, 5, 8, 1}
→ No swap
→ {2, 5, 1, 8}

Q.9) 6+12= 18

Q.10) Invalid because arr[10] is out of bounds.